

Algorithms and Data Structures

18.06.2025

Exercise 1 (1.5 points)

Consider a binary tree whose visits return the following sequences.

Pre-order:	A	E	F	I	K	J	B	C	H	D	G
In-order:	F	E	K	I	J	A	H	C	D	B	G
Post-order:	F	K	J	I	E	H	D	C	G	B	A

Report the sequence of keys stored on the tree leaves, moving on the tree from left to right. Please report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. The following is an example of the response format: A X C Y etc.

Exercise 2 (1.5 points)

Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=17$. Insertions occur character by character using open addressing with double hashing. Use functions $h_1(k) = k \% M$ and $h_2(k) = 1 + (k \% 97)$. Each character is identified by its index in the English alphabet (i.e., A=1, ..., Z=26). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

R H Y T H M

Indicate in which elements the last three letters of the sequence are placed, i.e., T, H, and M, in this order. Please report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response format: 3 4 11

Exercise 3 (4.0 points)

A set of strings, each one including an undefined number of any ASCII characters, are used as keys to access a hash table of size M, implemented with linear chaining. Report the C implementation of a proper hash function receiving the string as a parameter and returning the index of the hash table.

Describe the logic of the algorithm and specify how it is possible to avoid overflows during the computation of the hash function for long strings. Could you suggest a different implementation for the case in which the strings include only small alphabetic letters? Motivate all choices.

Exercise 4 (1.5 points)

Suppose we have an initially empty priority queue implemented with a maximum heap. Consider the following sequence of integers and "*" characters, where each integer corresponds to one insertion into the priority queue, and each character "*" corresponds to one extraction.

10 4 9 2 13 7 * *

Report the sequence of values as they are stored in the array representing the priority queue at the end of the entire process. Please show the entire array content as a sequence of integer values separated by a single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8 etc.

Exercise 5 (1.5 points)

The following capital letters are given with their absolute frequency.

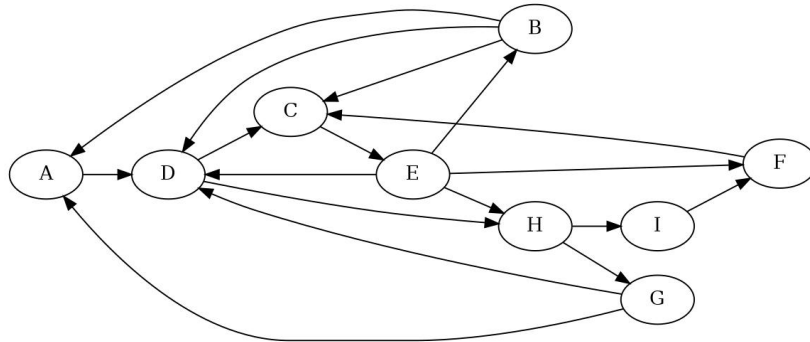
A:24 B:7 C:5 D:9 E:12 F:6

Find an optimal Huffman code for all symbols in the set using a greedy algorithm. Indicate which one of the following encoding is correct.

1. A(0) B(110) C(100) D(110) E(011) F(011)
2. A(00) B(01) C(10) D(110) E(101) F(1111)
3. A(01) B(10) C(110) D(110) E(110) F(001)
4. A(1) B(100) C(110) D(111) F(00) E(011)
5. A(01) B(100) C(110) D(111) E(10) F(00)
6. A(0) B(100) C(1100) D(101) E(111) F(1101)
7. A(10) B(100) C(110) D(100) E(110) F(00)
8. A(0) B(100) C(100) D(111) E(010) F(000)

Exercise 6 (1.5 points)

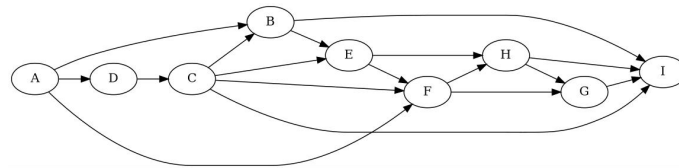
Visit the following graph in depth-first, starting at node A. Label each edge as tree (T), back (B), forward (F), and cross (C). When necessary, consider nodes and edges in alphabetical order.



Report the labels of the edges HI, IF, and DH in this order. Please indicate the edge type of these 3 arcs with a single letter, i.e., T, F, B, C, separated by single spaces. No other symbols must be included in the response. The following is an example of the response: B T C

Exercise 7 (1.5 points)

Given the following unweighted DAG, find the topological order of its vertices. Start the DFS visit from vertex A. If necessary, consider nodes in alphabetical order.



Report the direct topological order of the vertices. Report vertices on the same line separated by a single space. No other symbols must be included in the response. The following is an example of the response format: F A C H B etc.

Exercise 8 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```
#define N 4
void f (char m[N][N]);
int main() {
    char m[N][N] = {
        {'A', 'B', 'C', 'D'},
        {'E', 'F', 'G', 'H'},
        {'I', 'J', 'K', 'L'},
        {'M', 'N', 'O', 'P'}
    };
    f(m);
    return 0;
}

void f (char m[N][N]) {
    int t = 0, b = N - 1, l = 0, r = N - 1;
    while (t <= b && l <= r) {
        for (int i = l; i <= r; i++) {
            printf("%c ", m[t][i]);
        }
        t++;
        for (int i = t; i <= b; i++) {
            printf("%c ", m[i][r]);
        }
        r--;
        if (t <= b) {
            for (int i = r; i >= l; i--) {
                printf("%c ", m[b][i]);
            }
        }
    }
}
```

```

        b--;
    }
    if (l <= r) {
        for (int i = b; i >= t; i--) {
            printf("%c ", m[i][l]);
        }
        l++;
    }
}
}

```

Exercise 9 (2.0 points)

The following functions change the value of a key in a priority queue pq.

```

void pq_change (pq_t pq, int i, Item item) {
    if (item_less (item, pq->A[i]) {
        decrease_key (pq, i, item);
    } else {
        increase_key (pq, i, item);
    }
}

void decrease_key (pq_t pq, int i, Item item) {
    pq->A[i] = item;
    // LINE 1
}

void increase_key (pq_t pq, int I, Item item) {
    while( (i>=1) &&
           (item_less(pq->A[PARENT(i)], item)) ) {
        pq->A[i] = pq->A[PARENT(i)];
        //LINE 2
    }
    pq->A[i] = item;
}

```

Indicate which one of the following statements is correct. Note that incorrect answers imply a penalty in the final score.

1. In the function pq_change, functions increase_key and decrease_key must be swapped.
2. Line 1 must be: heapify (pq, i);
3. Line 1 must be: buildheap (pq, i);
4. Line 2 must be: i = PARENT(i);
5. Line 2 must be: i = RIGHT(i);
6. Line 2 must be: i = LEFT(i);
7. The cost of the function is linearitmic.
8. The cost of the function is logarithmic.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```

void f1(char [], int, int);
void f2(char [], int, int);
void f3(char [], int, int);

int main() {
    char *str = "HELLO";
    int index = 0;
    int count = 0;
    f1(str, index, count);
    return 0;
}

void f1(char str[], int index, int count) {
    if (count > strlen(str))
        return;
    index = (index + 1) % ((int) strlen(str));
    f2(str, index, count);
}

void f2(char str[], int index, int count) {
    printf("%c", str[index]);
    f3(str, index, count+1);
}

void f3(char str[], int index, int count) {
    index = (index - 2 + strlen(str)) % strlen(str);
    f1(str, index, count);
}

```

Exercise 11 (3.0 points)

Write the following C function:

```
int word_search (char **mat, int n, char *s);
```

That searches the string `s` into the square matrix `mat` of size `n`. The string must be stored in the matrix on contiguous characters, moving horizontally, vertically, or diagonally (again, only in consecutive positions). The function must return one (true) if the string is present, and zero (false) otherwise. For example, let us suppose that the input matrix `mat` of size four is the following:

a	b	c	e
x	e	c	s
t	d	e	f
x	y	v	w

If `s = "see"` is the string to search, it is present in the matrix in elements `[1,3]`, `[2,2]`, and `[1,1]`; thus, the function must return one.

Write the entire function using standard C libraries, but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 12 (5.0 points)

Write the following C function:

```
tree_t *convert_bst_to_min_heap (tree_t * root);
```

To convert a binary search tree (BST) referenced by the pointer `root` into a minimum heap. The function must return the root pointer to the heap.

Indicate the structure of type `tree_t` (that includes the two pointers and an integer key).

Hint: Traverse the BST to retrieve the elements in the correct order to be inserted into the heap in the first position available from top to bottom and left to right.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 13 (9.0 points)

On a chess board, a peculiar chess piece can move from one cell to another of a different color, i.e., from a white cell to a black one or vice versa. When it moves, it can reach cells at a distance of one (adjacent cells), at a distance of two (cells adjacent to the one adjacent), and cells at a distance of three. Initially, the piece is positioned in a specific cell of coordinates `[r,c]` in a square board of size `n`. Write the C function:

```
int visit (int **board, int n, int r, int c);
```

That makes the piece touch once and only once all the board's cells, starting from position `board[r][c]` so that the cost to move the piece is minimized. We can evaluate the cost of moving the piece as follows: Each move to a cell at a distance of 3 costs 0.1, each move to a cell at a distance 2 costs 0.2, and each move to a cell at a distance 1 costs 0.3. The entire path's cost is the sum of the cost of all singular moves.

In other words, it is necessary to find the path, including the maximum number of moves at a distance of three and the minimum number of moves at a distance of one.

The entire path's cost is the sum of the cost of all singular moves. In other words, it is necessary to find the path, including the maximum number of moves at a distance of three and the minimum number of moves at a distance of one. The function must assign to each cell of the board an integer value starting from zero (for cell `[r][c]`) and indicating for each cell the number of the moves performed to reach it within the path. Moreover, the function has to return a `float` value indicating the cost of such a path. If no path is found, the function must return zero, and the content of the board can be considered as undefined.

Write the entire function using standard C libraries, but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.